Universidad San Carlos de Guatemala

Facultad de Ingenieria

Manual Técnico / Práctica 2

Giovanni Josue Villegas Alvarez

202501086

Contents

Introduccion 3
Descipcion General 4
Recursividad 4
Uso de hilos 6
Interfaz gráfica 7
Visualización 7
Conclusión 8
Reflexión 9

Introduccion

En este proyecto se desarrolló una aplicación de escritorio en Java utilizando la librería Swing. El objetivo principal fue visualizar el funcionamiento de diferentes algoritmos de ordenamiento, permitiendo observar cómo los datos cambian paso a paso.

El programa incluye los algoritmos Bubble Sort, Shell Sort y Quick Sort, algunos en versión iterativa y otros en versión recursiva. Además, se implementó el uso de hilos para controlar la velocidad de ejecución y evitar que la interfaz se congele.

Este sistema ayuda a comprender mejor cómo funcionan los algoritmos, ya que no solo se ejecutan, sino que también se pueden visualizar de forma dinámica.

Descipcion General

El sistema consiste en una aplicación gráfica donde el usuario puede ingresar datos o generarlos de forma aleatoria, seleccionar un algoritmo de ordenamiento y observar cómo se ejecuta.

También cuenta con un modo de comparación, donde se ejecutan dos versiones de un mismo algoritmo al mismo tiempo para comparar su comportamiento.

El programa está dividido en diferentes módulos, como el de algoritmos, la interfaz gráfica, la visualización y el control de ejecución.

Algoritmos Implementados

- Bubble Sort Iterativo: Este algoritmo recorre el arreglo varias veces comparando elementos consecutivos e intercambiándolos si están en el orden incorrecto. Es sencillo de entender, pero no es el más eficiente.
- Bubble Sort Recursivo: En esta versión se aplica recursividad. Se realiza un recorrido del arreglo y luego el método se llama a sí mismo con un tamaño menor hasta llegar al caso base.
- Shell Sort: Este algoritmo mejora el Bubble Sort utilizando saltos (gaps) para comparar elementos más lejanos. Es más eficiente que Bubble Sort en la mayoría de casos.
- Quick Sort: Este algoritmo utiliza recursividad y divide el problema en partes más pequeñas. Selecciona un pivote y organiza los elementos alrededor de él. Es uno de los algoritmos más eficientes.

Recursividad

La recursividad es una técnica donde un método se llama a sí mismo. En este proyecto se utilizó en Bubble Sort y Quick Sort.

Cada algoritmo tiene un caso base, que indica cuándo detener la ejecución, y un caso recursivo, que es donde el método se vuelve a llamar.

Esto permite dividir el problema en partes más pequeñas hasta resolverlo completamente.

```java
 */
public class BubbleSortRecursivo {

    private OperationLogger logger;
    private long tiempoInicio;

    public BubbleSortRecursivo() {
        logger = new OperationLogger();
    }

    public void ordenar(int[] arreglo, boolean ascendente, int velocidad, SortingListener listener, SortingThread hilo) {
        logger.reiniciar();
        logger.iniciarTiempo();
        tiempoInicio = System.currentTimeMillis();

        bubbleSortRecursivo(arreglo, arreglo.length, ascendente, listener, hilo);

        logger.finalizarTiempo();

        if (listener != null) {
            listener.alActualizarArreglo(arreglo);
            listener.alRegistrarLog("Bubble Sort Recursivo finalizado.");
            listener.alActualizarEstadisticas(
                    logger.getComparaciones(),
                    logger.getIntercambios(),
                    logger.getLlamadasRecursivas(),
                    logger.getTiempoTotal()
            );
        }
    }

    private void bubbleSortRecursivo(int[] arreglo, int n, boolean ascendente, SortingListener listener, SortingThread hilo) {
        if (!hilo.isEjecutando()) {
            return;
        }

        logger.incrementarLlamadasRecursivas();

        if (n == 1) {
            return;
        }

        for (int j = 0; j < n - 1 && hilo.isEjecutando(); j++) {
            logger.incrementarComparaciones();

            if (listener != null) {
                listener.alComparar(arreglo, j, j + 1);
                listener.alRegistrarLog("Comparando posiciones " + j + " y " + (j + 1) + " en nivel recursivo n=" + n);
                listener.alActualizarEstadisticas(
```

```java
public class QuickSort {

    private OperationLogger logger;
    private long tiempoInicio;

    public QuickSort() {
        logger = new OperationLogger();
    }

    public void ordenar(int[] arreglo, boolean ascendente, int velocidad, SortingListener listener, SortingThread hilo) {
        logger.reiniciar();
        logger.iniciarTiempo();
        tiempoInicio = System.currentTimeMillis();

        quickSort(arreglo, 0, arreglo.length - 1, ascendente, listener, hilo);

        logger.finalizarTiempo();

        if (listener != null) {
            listener.alActualizarArreglo(arreglo);
            listener.alRegistrarLog("Quick Sort finalizado.");
            listener.alActualizarEstadisticas(
                    logger.getComparaciones(),
                    logger.getIntercambios(),
                    logger.getLlamadasRecursivas(),
                    logger.getTiempoTotal()
            );
        }
    }

    private void quickSort(int[] arreglo, int low, int high, boolean ascendente, SortingListener listener, SortingThread hilo) {
        if (!hilo.isEjecutando()) {
            return;
        }

        logger.incrementarLlamadasRecursivas();

        if (low < high) {
            int pi = partition(arreglo, low, high, ascendente, listener, hilo);

            quickSort(arreglo, low, pi - 1, ascendente, listener, hilo);
            quickSort(arreglo, pi + 1, high, ascendente, listener, hilo);
        }
    }
```

Uso de hilos

Se utilizaron hilos para controlar la ejecución de los algoritmos sin afectar la interfaz gráfica. Esto permite que el usuario pueda interactuar con el programa mientras el algoritmo se está ejecutando.

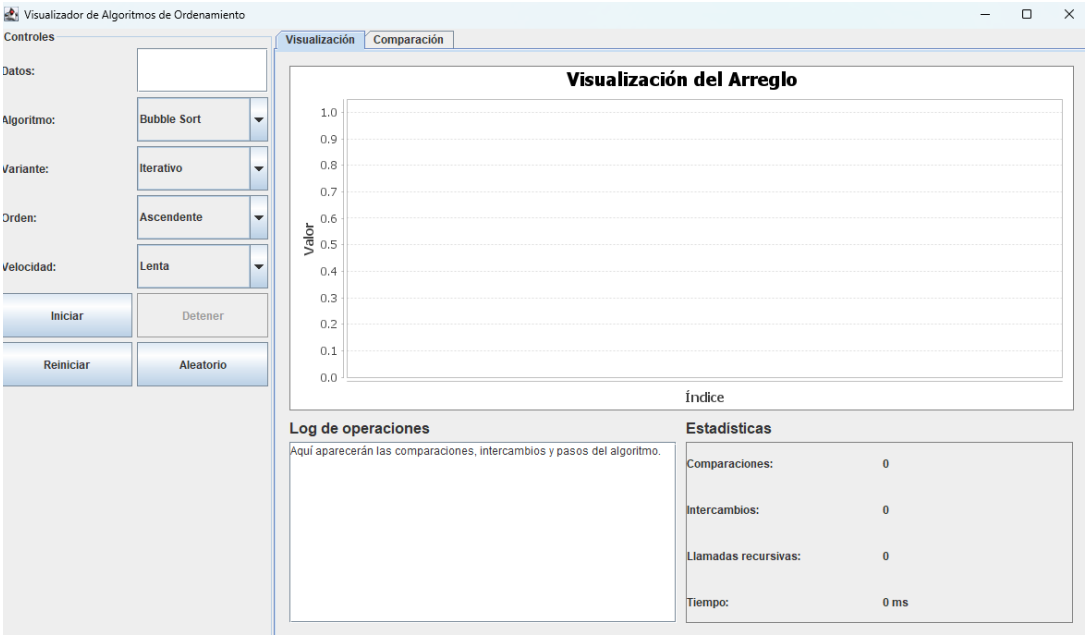
También se implementó el uso de diferentes velocidades, utilizando pausas entre cada paso del algoritmo.

```java
    public SortingThread(int[] arreglo, String algoritmo, String variante, boolean ascendente, int velocidad, SortingListener listener) {
        this.arreglo = arreglo;
        this.algoritmo = algoritmo;
        this.variante = variante;
        this.ascendente = ascendente;
        this.velocidad = velocidad;
        this.listener = listener;
        this.ejecutando = true;
        this.pausado = false;
    }

    @Override
    public void run() {
        try {
            if ("Bubble Sort".equals(algoritmo)) {
                if ("Iterativo".equals(variante)) {
                    BubbleSortIterativo bubbleIterativo = new BubbleSortIterativo();
                    bubbleIterativo.ordenar(arreglo, ascendente, velocidad, listener, this);
                } else if ("Recursivo".equals(variante)) {
                    BubbleSortRecursivo bubbleRecursivo = new BubbleSortRecursivo();
                    bubbleRecursivo.ordenar(arreglo, ascendente, velocidad, listener, this);
                }
            } else if ("Shell Sort".equals(algoritmo)) {
                ShellSort shellSort = new ShellSort();
                shellSort.ordenar(arreglo, ascendente, velocidad, listener, this);
            } else if ("Quick Sort".equals(algoritmo)) {
                QuickSort quickSort = new QuickSort();
                quickSort.ordenar(arreglo, ascendente, velocidad, listener, this);
            }

            if (listener != null && ejecutando) {
                listener.alFinalizar(arreglo);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public boolean isEjecutando() {
        return ejecutando;
    }

    public void detenerEjecucion() {
        ejecutando = false;
        reanudarEjecucion();
    }
```

Interfaz gráfica

La interfaz fue desarrollada con Java Swing. Se diseñó una ventana principal que contiene los controles del sistema y la visualización del arreglo.

El usuario puede seleccionar el algoritmo, la velocidad y el tipo de ordenamiento. También puede iniciar, detener o reiniciar la ejecución.



Visualización

Se utilizó una librería de gráficos para mostrar el arreglo en forma de barras. Cada barra representa un valor.

Los colores indican lo que está sucediendo en el algoritmo, por ejemplo:

- Comparación

- Intercambio

- Elementos normales

Conclusión

Este proyecto permitió comprender mejor el funcionamiento de los algoritmos de ordenamiento, así como la diferencia entre iteración y recursividad.

Además, se aprendió a trabajar con interfaces gráficas y a utilizar hilos para mejorar la experiencia del usuario.

Reflexión

Durante el desarrollo de este proyecto pude observar de forma clara las diferencias entre la programación iterativa y la recursiva, especialmente al trabajar con el algoritmo Bubble Sort en el modo comparación. La versión iterativa funciona mediante ciclos repetitivos, lo cual la hace más directa de entender, ya que sigue un proceso paso a paso bastante claro. En cambio, la versión recursiva divide el problema en partes más pequeñas y se llama a sí misma, lo que al inicio puede resultar un poco más difícil de comprender, pero al verlo de forma visual se vuelve mucho más claro.

Al ejecutar ambas versiones al mismo tiempo, se puede notar que aunque hacen prácticamente lo mismo, la forma en que trabajan internamente es diferente. La versión recursiva depende de la pila de llamadas, mientras que la iterativa utiliza estructuras como ciclos. Esto me ayudó a entender mejor cómo funciona la recursividad en la práctica y no solo en teoría.

En el caso de Quick Sort, es evidente por qué es un algoritmo naturalmente recursivo. Esto se debe a que divide el arreglo en subarreglos más pequeños alrededor de un pivote, y luego aplica el mismo proceso a cada uno de esos subproblemas. Este tipo de solución encaja perfectamente con la recursividad, ya que el problema se resuelve repitiendo la misma lógica en partes más pequeñas.

En cuanto a las ventajas y desventajas, considero que la recursividad puede ser más elegante y fácil de escribir en ciertos casos, como Quick Sort, pero también puede ser más difícil de entender y consumir más recursos si no se controla bien. Por otro lado, la iteración suele ser más sencilla y eficiente en algunos casos, pero puede volverse más larga o repetitiva.

En general, este proyecto me permitió comprender mejor estos conceptos al verlos en funcionamiento, lo cual considero mucho más útil que solo estudiarlos de forma teórica.